

CM02 - Bash Scripting

Builtins, processes, variables, scripts, conditions, loops, functions

Louis Ledoux

2026-07-05

Command Implementation and Processes

Command Implementation and Processes

Live pacing: about 8 min

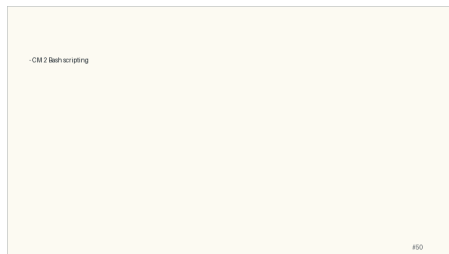
Question. When does the shell execute commands, and when does it just transform text?

Core claim. A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.

Target capability. Students can read a script by following expansion, command status, branches, loops, and functions.

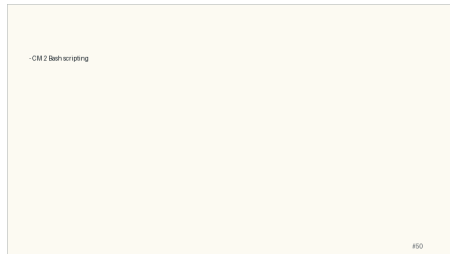
Main Path

- ▶ Start from the question: When does the shell execute commands, and when does it just transform text?
- ▶ Build the model: A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.
- ▶ End with a checkable action: Students can read a script by following expansion, command status, branches, loops, and functions.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Trace one script twice: once with existing files, once with a missing file, and compare statuses.
- ▶ Representative source slide: 50
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.



Anecdote Or Motivation

Shell is powerful because it glues existing programs; it is dangerous for the same reason.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Process management Control execution

- End a running process
- Global method:
- kill PID
- Foreground execution in current shell (through SIGNALS)
- CTRL + C (Interrupt)
- CTRL + D (EOF)
- CTRL + \ (Quit)
- Pause a process
- CTRL + Z
- Resume
- Foreground (fg)
- Background (bg)

#98

Backup Index

Original slides 50-58

-CM 2 Bash scripting

Backup: CM 2 Bash scripting

Original slide 50

-CM 2 Bash scripting

Backup: Command implementation

Original slide 51

- ▶ Builtin
- ▶ Implemented in the code of the shell
- ▶ They affect the internal state of the shell (ex. `cd`)
- ▶ External
- ▶ Implemented as a separate code
- ▶ Shell interprets the command as a request to load and run the program that implements the command
- ▶ Stored in `/bin`

Command implementation

- Builtin
 - implemented in the code of the shell
 - they affect the internal state of the shell (ex. `cd`)
- External
 - implemented as a separate code
 - Shell interprets the command as a request to load and run the program that implements the command
 - stored in `/bin`

451

Backup: Builtin Commands

Original slide 52

Builtin Commands

Backup: Builtin Commands

Original slide 53

Builtin Commands

Backup: Process management: Execution

Original slide 54

- ▶ Process: The running instance of a program, e.g.:
- ▶ Execution of a command
- ▶ Execution of your program
- ▶ Foreground execution: Interactive processes
- ▶ Connected to a shell terminal
- ▶ User can send input
- ▶ Background execution: Non-Interactive or automated processes
- ▶ Not connected to a terminal
- ▶ User cannot send input until the process is moved to the foreground

Process management: Execution

- Process: The running instance of a program, e.g.
 - Execution of a command
 - Execution of your program
- Foreground execution: Interactive processes
 - Connected to shell terminal
 - User can send input
- Background execution: Non-interactive or automated processes
 - Not connected to a terminal
 - User cannot send input until the process is moved to the foreground

454

Backup: Process management: Changing grounds

Original slide 55

- ▶ Run in Background: `&`:
- ▶ Ex: `gedit &`
- ▶ From FG to BG:
- ▶ Ex: `gedit`
- ▶ Press `CTRL + Z` (puts process in idle state)
- ▶ `bg`
- ▶ From BG to FG:
- ▶ Ex: `fg %jobid`
- ▶ (jobid is not PID. It is a unique number given by the shell, e.g. when process goes to BG)

Process management: Changing grounds

- Run in Background: `&`
- Ex: `gedit &`
- From FG to BG:
- Ex: `gedit`
- Press `CTRL + Z` (puts process in idle state)
- `bg`
- From BG to FG:
- Ex: `fg %jobid`
- (jobid is not PID. It is a unique number given by the shell, e.g. when process goes to BG)

455

Backup: Process management

Original slide 56

- ▶ Process: The running instance of a program, e.g.:
- ▶ Execution of a command
- ▶ Execution of your program
- ▶ Serial execution: ;
- ▶ All commands are executed
- ▶ New prompt when last command finishes
- ▶ Ex: `who | sort ; date`
- ▶ Parallel execution: &
- ▶ All commands are executed
- ▶ A new prompt is directly available (execution at the background)
- ▶ Ex: `pr junk | lpr &`

Process management

- Process: The running instance of a program, e.g.
- Execution of a command
- Execution of your program
- Serial execution: ;
- All commands are executed
- New prompt when last command finishes
- Ex: `who | sort ; date`
- Parallel execution: &
- All commands are executed
- A new prompt is directly available (execution at the background)

450

Backup: Process management: Active processes

Original slide 57

- ▶ Print the catalogue of the running processes: `ps`
- ▶ `-a`: Select all processes associated with a terminal.
- ▶ PID: Unique identity of the process
- ▶ TTY: Console created the process
- ▶ TIME: Total execution time (sec)
- ▶ CMD: Command executed

Process management: Active processes

- Print the catalogue of the running processes: `ps`
- `-a`: Select all processes associated with a terminal.
- PID: Unique identity of the process
- TTY: Console created the process
- TIME: Total execution time (sec)
- CMD: Command executed

```
engine@chillians:~/Downloads$ ps
  PID TTY          TIME CMD
 32598 pts/1    00:00:00 bash
 34410 pts/1    00:00:00 vi
 34447 pts/1    00:00:00 evince
 34454 pts/1    00:00:00 ps
```

457

Backup: Process management: Control execution

Original slide 58

- ▶ End a running process
- ▶ Global method:
- ▶ kill PID
- ▶ Foreground execution in current shell (through SIGNALS)
- ▶ CTRL + C (Interrupt)
- ▶ CTRL + D (EOF)
- ▶ CTRL + \ (Quit)
- ▶ Pause a process
- ▶ CTRL + Z
- ▶ Resume
- ▶ Foreground (fg)
- ▶ Background (bg)

Process management: Control execution

- End a running process
 - Global method
 - kill PID
- Foreground execution in current shell (through SIGNALS)
 - CTRL + C (Interrupt)
 - CTRL + D (EOF)
 - CTRL + \ (Quit)
- Pause a process
 - CTRL + Z
- Resume
 - Foreground (fg)
 - Background (bg)

498

Shell Variables and Scripts

Shell Variables and Scripts

Live pacing: about 8 min

Question. When does the shell execute commands, and when does it just transform text?

Core claim. A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.

Target capability. Students can read a script by following expansion, command status, branches, loops, and functions.

Main Path

- ▶ Start from the question: When does the shell execute commands, and when does it just transform text?
- ▶ Build the model: A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.
- ▶ End with a checkable action: Students can read a script by following expansion, command status, branches, loops, and functions.
- ▶ Keep only this slide on the horizontal path during the live lecture.

Shell Variables

- Initial values defined in system and user configuration file
- Use uppercase by convention
- Definition of variables based on the type (bash shell)
- String variables: MYVAR="value" (NO SPACES around =)
- Integer variables: declare -i MYVAR
- Constant variables: readonly DATA=value
- Array variables: declare -a MYARRAY MYARRAY[0]="one" MYARRAY[1]=5
- Use of variables: \$
- E.g. \$MYVAR
- Lifetime of variables

#99

Worked Example

- ▶ Use this as the live trace: Trace one script twice: once with existing files, once with a missing file, and compare statuses.
- ▶ Representative source slide: 64
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Script: Execution

No need for permissions:
bash path, to script/mycscript.sh
bash command is executed
path, to script/mycscript.sh is the command-line argument
Provide execute permissions and execute:
chmod +x mycscript.sh
./mycscript.sh
Shell forks itself and uses a system call (e.g. execve) to make the OS execute the file in the
forked process.
OS checks the file's permissions

#64

Anecdote Or Motivation

Shell is powerful because it glues existing programs; it is dangerous for the same reason.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Arithmetic operations

- Arithmetic operations with integers
- let (without spaces as it is an identifier)
- expr (with spaces in arguments)



```
user> let x = 10
user> let y = 20
user> let z = x + y
user> expr x
10
user> expr y
20
user> expr z
30
```

#70

Backup Index

Original slides 59-70

Shell Variables

- Initial values: defined in system and user configuration file
- Use uppercase by convention
- Definition of variables based on the type (bash shell)
- String variables: MYVAR="value" (NO SPACES around =)
- Integer variables: declare -i MYVAR
- Constant variables: readonly DATA=value
- Array variables: declare -a MYARRAY MYARRAY[0]="one"; MYARRAY[1]=5
- Use of variables: \$
- E.g. \$MYVAR
- Lifetime of variables:

Backup: Shell Variables

Original slide 59

- ▶ Initial values: defined in system and user configuration file
- ▶ Use uppercase by convention
- ▶ Definition of variables based on the type (bash shell)
- ▶ String variables: MYVAR="value" (NO SPACES around =)
- ▶ Integer variables: declare -i MYVAR
- ▶ Constant variables: readonly DATA=value
- ▶ Array variables: declare -a MYARRAY
MYARRAY[0]="one"; MYARRAY[1]=5
- ▶ Use of variables: \$
- ▶ E.g. \$MYVAR
- ▶ Lifetime of variables:
- ▶ End of session - termination of shell
- ▶ Deleted by user: unset VARNAME

Shell Variables

- Initial values defined in system and user configuration file
- Use uppercase by convention
- Definition of variables based on the type (bash shell)
- String variables: MYVAR="value" (NO SPACES around =)
- Integer variables: declare -i MYVAR
- Constant variables: readonly DATA=value
- Array variables: declare -a MYARRAY MYARRAY[0]="one" MYARRAY[1]=5
- Use of variables: \$
- E.g. \$MYVAR
- Lifetime of variables

490

Backup: Shell Variables: Scope

Original slide 60

- ▶ Global: Public variable seen by all subshells
- ▶ Set export attitude for the variable: `export`
- ▶ Print global variables: `env`, `printenv`
- ▶ Local variables: Seen by current shell
- ▶ Print list of local and global variables: `set`

Shell Variables: Scope

- Global: Public variable seen by all subshells
- Set export attitude for the variable: `export`
- Print global variables: `env`, `printenv`
- Local variables: Seen by current shell
- Print list of local and global variables: `set`

#60

Backup: Shell Variables: Scope example

Original slide 61

Shell Variables: Scope example

```
angie@achilleas:~$ myvar="hello"
angie@achilleas:~$ set | grep myvar
myvar=hello
angie@achilleas:~$ echo "$$"
12190
angie@achilleas:~$ bash
angie@achilleas:~$ echo "$$"
15752
angie@achilleas:~$ set | grep myvar
angie@achilleas:~$ env | grep myvar
angie@achilleas:~$ exit
exit
angie@achilleas:~$ export myvar="hello"
angie@achilleas:~$ bash
angie@achilleas:~$ echo "$$"
15825
angie@achilleas:~$ set | grep myvar
myvar=hello
angie@achilleas:~$ █
```

Backup: Predefined shell variables

Original slide 62

Predefined shell variables

Backup: Scripts: Structure

Original slide 63

Program consisting of commands:

Use an editor:

`vi` (vim), emacs, sublime, gedit, ...

1st line defines the shell to be used:

`#!/bin/bash` (for bash shell)

Write program commands

Commands can be separated by:

a new line

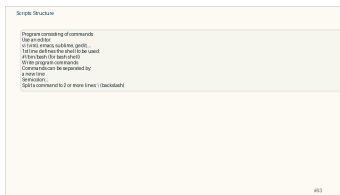
Semicolon: ;

Split a command to 2 or more lines: \ (backslash)

For comments: #

Save file with extension .sh

The script lines (commands) are interpreted and NOT compiled



Backup: Scripts: Execution

Original slide 64

No need for permissions:

```
bash path_to_script/myscript.sh
```

bash command is executed

path_to_script/myscript.sh is the command-line arg

Provide eXecute permissions and execute:

```
chmod +x myscript.sh
```

```
./myscript.sh
```

Shell forks itself and uses a system call (e.g. `execve`) to make the OS execu

OS checks the file's permissions

Program loader looks at the file to find how to execute it

(`#!` for scripts and then which interpreter to use: `/bin/bash`).

New process: attention to the variables and functions

Script Execution

No need for permissions
bash path_to_script/myscript.sh
bash command is executed
path_to_script/myscript.sh is the command-line argument
Provide eXecute permissions and execute:
chmod +x myscript.sh
./myscript.sh

Shell forks itself and uses a system call (e.g. `execve`) to make the OS execute the file in the background.
OS checks the file's permissions

#64

Backup: Command-line arguments

Original slide 65

Arguments can be passed along with the script execution

```
./myscript.sh arg1 ... argN
```

The arguments are seen as specific variables in the script

\$0: Name of the script

\$1: 1st argument

\$N: Nth argument

\$#: Number of arguments

\$*: All the arguments passed to the script, as a string

\$@: Behaves like \$* except that when the arguments are quoted, they are broken up properly

Maximum length of arguments is defined by operating system and measured in Kbytes
getconf ARG_MAX

Command-line arguments

Arguments can be passed along with the script execution
./myscript.sh arg1 ... argN
The arguments are seen as specific variables in the script code
\$0: Name of the script
\$1: 1st argument
\$N: Nth argument
\$#: Number of arguments
\$*: All the arguments passed to the script, as a string
\$@: Behaves like \$* except that when the arguments are quoted, they are broken up properly if there are spaces in them

#65

Backup: Variables

Original slide 66

```
Initialization: nomvar=expression
ATTENTION! No space around =
expression:
Value
Reference to a variable
`command` (replaced with the result to the stdout)
Use: $nomvar
Print current directory
y=`pwd`
echo $y
Print the first two arguments of a script
y="$1 and $2"
echo $y
```



Backup: Read input

Original slide 67

- ▶ We can ask the user to provide some value(s) to the variables
- ▶ `read var-name1 var-name2 ... var-nameN`
- ▶ Reads a line from the stdin and affects the first value to `var-name1`, the second value to `var-name2`, etc.
- ▶ Returns 0, if the read was successful
- ▶ We can print a message to the user:
- ▶ `read -p "Enter value:" var`
- ▶ If variable is not specified, then the default is used:
- ▶ `$REPLY`

Read input

- We can ask the user to provide some value(s) to the variables
- `read var-name1 var-name2 ... var-nameN`
- Reads a line from the stdin and affects the first value to `var-name1`, the second value to `var-name2`, etc.
- Returns 0, if the read was successful
- We can print a message to the user:
- `read -p "Enter value:" var`
- If variable is not specified, then the default is used
- `$REPLY`

67

Backup: Print to output

Original slide 68

- ▶ Basic print to stdout: echo
- ▶ Strings
- ▶ Variables
- ▶ Wildcards characters
- ▶ Can have several lines

Print to output

- Basic: print to stdout: echo
- Strings
- no spaces
- Wildcards characters
- Can have several lines

```
angiedachillias:~/Entertainment$ echo hello
hello
angiedachillias:~/Entertainment$ echo *
movies Music photos
angiedachillias:~/Entertainment$ echo print '% user'
print % user
angiedachillias:~/Entertainment$ echo 'hell
o o there'
hell
o there
```

#68

Backup: Arithmetic operations

Original slide 69

- ▶ No need to define the variables as integers
- ▶ Arithmetic operations with integers
- ▶ arithmetic expansion: expression evaluation and substitution with the result
- ▶ `$((...))`: Portability among shells
- ▶ `((...))`

Arithmetic operations

- No need to define the variables as integers
- Arithmetic operations with integers
- Arithmetic expression: expression evaluation and substitution with the result
- `if`, `if`, `if` Portability among the lis
- `if`, `if`



```
1 // Arithmetic operations
2
3 #include <stdio.h>
4
5 int main()
6 {
7     int a = 10;
8     int b = 20;
9     int c = a + b;
10    printf("Sum of %d and %d is %d\n", a, b, c);
11    return 0;
12 }
```

2/3

Backup: Arithmetic operations

Original slide 70

- ▶ Arithmetic operations with integers
- ▶ `let` (without spaces as it is an initialisation)
- ▶ `expr` (with spaces in arguments)



Conditions

Conditions

Live pacing: about 8 min

Question. When does the shell execute commands, and when does it just transform text?

Core claim. A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.

Target capability. Students can read a script by following expansion, command status, branches, loops, and functions.

Main Path

- ▶ Start from the question: When does the shell execute commands, and when does it just transform text?
- ▶ Build the model: A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.
- ▶ End with a checkable action: Students can read a script by following expansion, command status, branches, loops, and functions.
- ▶ Keep only this slide on the horizontal path during the live lecture.

Condition expressions: test

- A condition expression is:
 - a set of commands that are executed
 - The test command exits with the exit status determined by the condition_expression
- test condition_expression
- [condition_expression]
- [[condition_expression]]
- Similar to single brackets but more powerful
- ((arithmetic_condition_expression))
- (command)

- SPACES ARE REQUIRED !!

Worked Example

- ▶ Use this as the live trace: Trace one script twice: once with existing files, once with a missing file, and compare statuses.
- ▶ Representative source slide: 73
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Exit status

Syntax
\$1
Function
If the command ends correctly (TRUE) exits with value 0 (exit 0)
If there was an error (FALSE) exits with value 1 (exit 1)
Commands that do nothing except return an exit status of
true: Return an exit status of zero
false: Return an exit status of one

- ATTENTION !!

```
$ true; echo $?  
$ 0  
$ false; echo $?  
$ 1
```

#73

Anecdote Or Motivation

Shell is powerful because it glues existing programs; it is dangerous for the same reason.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Conditions case Example

```
#!/bin/bash
read -p "Enter a command: " command
case $command in
  all | ALL)
    echo "Display all files."
    ls -la;
  list | LIST)
    echo "Display all non-hidden files."
    ls -li;
  *)
    echo "Invalid choice"
    esac
```

#02

Backup Index

Original slides 71-82

Condition expressions: test

- A condition expression is
- a set of commands that are executed
- The test command exits with the exit status determined by the condition_expression
- test condition_expression
- [condition_expression]
- [[condition_expression]]
- Similar to single brackets, but more powerful
- ((arithmetic condition_expression))
- { command }

- SPACES ARE REQUIRED !!

Backup: Condition expressions: test

Original slide 71

- ▶ A condition expression is:
- ▶ a set of commands that are executed
- ▶ The test command exits with the exit status determined by the `condition_expression`
- ▶ `test condition_expression`
- ▶ `[ condition_expression ]`
- ▶ `[[ condition_expression ]]`
- ▶ Similar to single brackets, but more powerful
- ▶ `(( arithmetic condition_expression ))`
- ▶ `( command )`
- ▶ SPACES ARE REQUIRED !!

Condition expressions: test

- A condition expression is
 - a set of commands that are executed
 - The test command exits with the exit status determined by the `condition_expression`
- `test condition_expression`
- `[ condition_expression ]`
- `[[ condition_expression ]]`
- Similar to single brackets, but more powerful
- `(( arithmetic condition_expression ))`
- `( command )`

- SPACES ARE REQUIRED !

471

Backup: Condition expressions: test operators

Original slide 72

Condition expressions: test operators

Backup: Exit status

Original slide 73

Syntax

\$?

Function

If the command ends correctly (TRUE): exits with value 0

If there was an error (FALSE): exits with value 1

Commands that do nothing except return an exit status of:

true: Return an exit status of zero

false: Return an exit status of one

► ATTENTION !!

```
$ true ; echo $?
```

```
$ 0
```

```
$ false ; echo $?
```

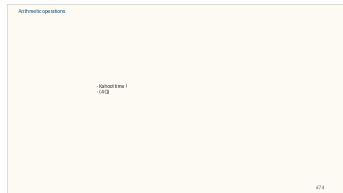
```
$ 1
```



Backup: Arithmetic operations

Original slide 74

- ▶ Kahoot time !
- ▶ (4 Q)



Backup: Boolean control operators

Original slide 75

- ▶ Syntax (AND): -a or &&
- ▶ `command1 && command2`
- ▶ Function:
- ▶ `command2` is executed if, and only if, `command1` returns an exit status of zero (TRUE).
- ▶ Syntax (OR): -o or ||
- ▶ `command1 || command2`
- ▶ Function:
- ▶ `command2` is executed if, and only if, `command1` returns a non-zero exit status (FALSE).
- ▶ The exit status of AND and OR lists is the exit status of the last command executed in the list
- ▶ Syntax (NOT): !

Boolean control operators

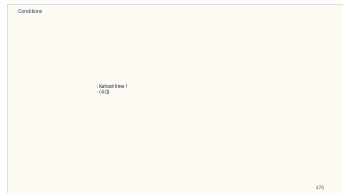
- Syntax (AND): -a or &&
- `command1 && command2`
- Function:
- `command2` is executed if, and only if, `command1` returns an exit status of zero (TRUE).
- Syntax (OR): -o or ||
- `command1 || command2`
- Function:
- `command2` is executed if, and only if, `command1` returns a non-zero exit status (FALSE).
- The exit status of AND and OR lists is the exit status of the last command executed in the list.
- Syntax (NOT): !

475

Backup: Conditions

Original slide 76

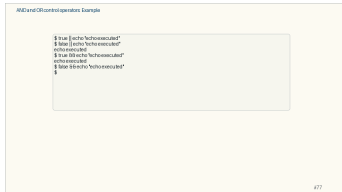
- ▶ Kahoot time !
- ▶ (4 Q)



Backup: AND and OR control operators: Example

Original slide 77

```
$ true || echo "echo executed"
$ false || echo "echo executed"
echo executed
$ true && echo "echo executed"
echo executed
$ false && echo "echo executed"
$
```



Backup: Conditions: if, elif, else

Original slide 78

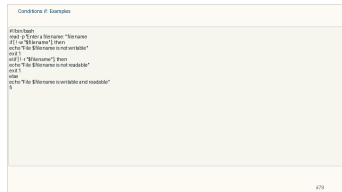
- ▶ Syntax:
- ▶ if [condition_expression_1] ; then
- ▶ 1st set of commands
- ▶ elif [condition_expression_2]
- ▶ then
- ▶ 2nd set of commands
- ▶ else
- ▶ 3rd set of commands
- ▶ fi
- ▶ Function
- ▶ The if checks if the condition_expression_1 returns TRUE
- ▶ 1st set of commands is executed
- ▶ Else, if the condition_expression_2 returns TRUE
- ▶ 2nd set of commands is executed
- ▶ Otherwise, 3rd set of commands is executed



Backup: Conditions: if : Examples

Original slide 79

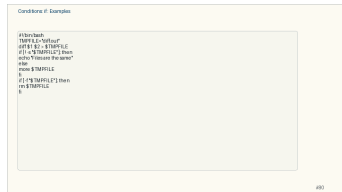
```
#!/bin/bash
read -p "Enter a filename: " filename
if [ ! -w "$filename" ]; then
echo "File $filename is not writable"
exit 1
elif [ ! -r "$filename" ]; then
echo "File $filename is not readable"
exit 1
else
echo "File $filename is writable and readable"
fi
```



Backup: Conditions: if : Examples

Original slide 80

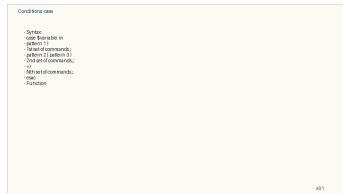
```
#!/bin/bash
TMPFILE="diff.out"
diff $1 $2 > $TMPFILE
if [ ! -s "$TMPFILE" ]; then
echo "Files are the same"
else
more $TMPFILE
fi
if [ -f "$TMPFILE" ]; then
rm $TMPFILE
fi
```



Backup: Conditions: case

Original slide 81

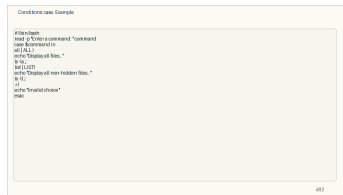
- ▶ Syntax:
- ▶ case \$variable in
- ▶ pattern 1)
- ▶ 1st set of commands ;;
- ▶ pattern 2 | pattern 3)
- ▶ 2nd set of commands ;;
- ▶ *)
- ▶ Nth set of commands ;;
- ▶ esac
- ▶ Function
- ▶ If the value of variable is pattern 1, the 1st set of commands is executed, if it is pattern 2, the 2nd set of commands is executed etc.
- ▶ Otherwise (*) -> Nth set of commands



Backup: Conditions: case: Example

Original slide 82

```
#!/bin/bash
read -p "Enter a command: " command
case $command in
all | ALL )
echo "Display all files..."
ls -la ;;
list | LIST)
echo "Display all non-hidden files..."
ls -ll ;;
*)
echo "Invalid choice"
esac
```



Iterations

Iterations

Live pacing: about 8 min

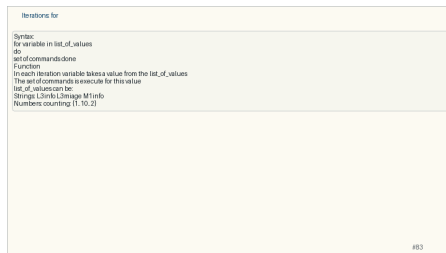
Question. When does the shell execute commands, and when does it just transform text?

Core claim. A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.

Target capability. Students can read a script by following expansion, command status, branches, loops, and functions.

Main Path

- ▶ Start from the question: When does the shell execute commands, and when does it just transform text?
- ▶ Build the model: A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.
- ▶ End with a checkable action: Students can read a script by following expansion, command status, branches, loops, and functions.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Trace one script twice: once with existing files, once with a missing file, and compare statuses.
- ▶ Representative source slide: 83
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Iterations for

Syntax:
for variable in list_of_values
do
set of commands done
Function
In each iteration variable takes a value from the list_of_values
The set of commands to execute for this value
list_of_values can be:
Strings L3mfo L3maga M1mfo
Numbers counting [1, 10, 2]

#83

Anecdote Or Motivation

Shell is powerful because it glues existing programs; it is dangerous for the same reason.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Controlling loop commands Example

```
- x is 1. Most numbers get here, except 4.  
- x is 2. Most numbers get here, except 4.  
- x is 3. Most numbers get here, except 4.  
- x is 4.  
- x is 5. Most numbers get here, except 4.  
- x is 6. Most numbers get here, except 4.  
- x is 7. Most numbers get here, except 4.  
- We have finished the loop now.  
- At the end, x is now 7.
```

#91

Backup Index

Original slides 83-91

Iterations: for

Syntax:

for variable in list_of_values

do

set of commands done

Function

In each iteration variable takes a value from the list_of_values

The set of commands is execute for this value

list_of_values can be:

Strings: L3info L3miage M1info

Numbers: counting: {1..10..2}

Backup: Iterations: for

Original slide 83

Syntax:

```
for variable in list_of_values  
do
```

set of commands done

Function

In each iteration variable takes a value from the list_of_values

The set of commands is execute for this value

list_of_values can be:

Strings: L3info L3miage M1info

Numbers: counting: {1..10..2}

enumerating: 1 3 5

Command line arguments: \$*

File names: filename1 filename2

command output: \$(ls /tmp/*)

Iterations for

Syntax
for variable in list_of_values
do
set of commands done
Function
In each iteration variable takes a value from the list_of_values
The set of commands is execute for this value
list_of_values can be:
Strings: L3info L3miage M1info
Numbers: counting {1..10..2}

83

Backup: Iterations: for: Examples

Original slide 84

```
#!/bin/bash
for file in f1 f2 f3
do
  cp $file $1/$file
done
```

```
#!/bin/bash
for i in 6 3 1 2
do
  echo $i
done | sort -n
```

Iterations for Examples

```
#!/bin/bash
for file in f1 f2 f3
do
  cp $file $1/$file
done
```

```
#!/bin/bash
for i in 6 3 1 2
do
  echo $i
done | sort -n
```

84

Backup: Iterations: while

Original slide 85

- ▶ Syntax:
- ▶ while
- ▶ [condition_expression]
- ▶ do
- ▶ set of commands
- ▶ done
- ▶ Function
- ▶ As long as the condition_expression returns TRUE
- ▶ The set_of_commands is executed
- ▶ Otherwise, the iteration ends

Iterations: while

- Syntax:
 - while
 - (condition_expression)
 - do
 - set of commands
 - done
- Function
 - As long as the condition_expression returns TRUE
 - The set_of_commands is executed
 - Otherwise, the iteration ends

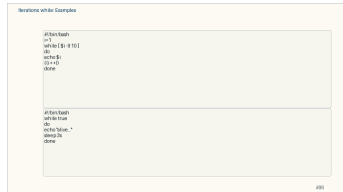
85

Backup: Iterations: while: Examples

Original slide 86

```
#!/bin/bash
i=1
while [ $i -lt 10 ]
do
echo $i
((i++))
done
```

```
#!/bin/bash
while true
do
echo "alive..."
sleep 3s
done
```



Backup: Iterations: until

Original slide 87

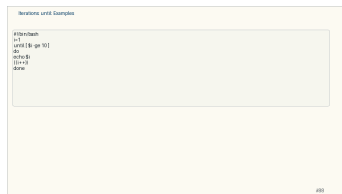
- ▶ Syntax:
- ▶ until
- ▶ [condition_expression]
- ▶ do
- ▶ set of commands
- ▶ done
- ▶ Function
- ▶ Execute 1st set of commands
- ▶ As long as the condition_expression commands is NOT true (while condition_expression is FALSE)
- ▶ The set_of_commands is executed
- ▶ Otherwise the iteration ends



Backup: Iterations: until: Examples

Original slide 88

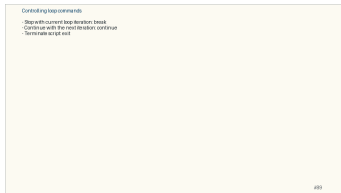
```
#!/bin/bash
i=1
until [ $i -ge 10 ]
do
echo $i
((i++))
done
```



Backup: Controlling loop commands

Original slide 89

- ▶ Stop with current loop iteration: `break`
- ▶ Continue with the next iteration: `continue`
- ▶ Terminate script: `exit`



Backup: Controlling loop commands: Example

Original slide 90

```
#!/bin/bash
for x in 1 2 3 4 5 6 7 8 9
do
echo -n "x is $x. "
if [ "$x" -eq "4" ]; then
continue
fi
echo "Most numbers get here, except 4."
if [ "$x" -eq "7" ]; then
break
fi
done
echo "We have finished the loop now."
echo "At the end, x is now $x."
```



Backup: Controlling loop commands: Example

Original slide 91

- ▶ x is 1. Most numbers get here, except 4.
- ▶ x is 2. Most numbers get here, except 4.
- ▶ x is 3. Most numbers get here, except 4.
- ▶ x is 4.
- ▶ x is 5. Most numbers get here, except 4.
- ▶ x is 6. Most numbers get here, except 4.
- ▶ x is 7. Most numbers get here, except 4.
- ▶ We have finished the loop now.
- ▶ At the end, x is now 7.



Functions

Functions

Live pacing: about 8 min

Question. When does the shell execute commands, and when does it just transform text?

Core claim. A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.

Target capability. Students can read a script by following expansion, command status, branches, loops, and functions.

Main Path

- ▶ Start from the question: When does the shell execute commands, and when does it just transform text?
- ▶ Build the model: A shell script is a small program whose data model is arguments, variables, statuses, files, and streams.
- ▶ End with a checkable action: Students can read a script by following expansion, command status, branches, loops, and functions.
- ▶ Keep only this slide on the horizontal path during the live lecture.

Functions

```
Syntax:
function name()
{
  list of commands
  (return)
}
Function
Functions have to be defined in the beginning of the script
They may have or not arguments
Arguments and return values can be of any type
The variables defined inside the function are GLOBAL. They have to be defined LOCAL, if we want
```

#92

Worked Example

- ▶ Use this as the live trace: Trace one script twice: once with existing files, once with a missing file, and compare statuses.
- ▶ Representative source slide: 92
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Functions

```
Syntax
function name{()
{
  list of commands
}
return()
```

Function

Functions have to be defined in the beginning of the script

They may have or not arguments

Arguments and return values can be of any type

The variables defined inside the function are GLOBAL. They have to be defined LOCAL, if we want

#92

Anecdote Or Motivation

Shell is powerful because it glues existing programs; it is dangerous for the same reason.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Functions Example

```
#!/bin/bash
outside="a global variable"
function mine() {
  local inside="this is local"
  echo $outside
  echo $inside
  outside="a global with new value"
}
echo $outside
mine
echo $outside
echo $inside
```

- a global variable
- a global variable
- this is local
- a global with new value

#93

Backup Index

Original slides 92-93

Functions

Syntax

```
function name[()]
```

```
{
```

```
list of commands
```

```
[return]}
```

Function

Functions have to be defined in the beginning of the script

They may have or not arguments

Arguments and return values can be of any type

The variables defined inside the function are GLOBAL. They have to be defined LOCAL, if we want

Backup: Functions

Original slide 92

Syntax

```
function name[()]\n{\nlist of commands;\n[return]}
```

Function

Functions have to be defined in the beginning of the script

They may have or not arguments

Arguments and return values can be of any type

The variables defined inside the function are GLOBAL. They have to be defined

Functions

Syntax

function name()
{
 list of commands;
 [return];
}
Function
Functions have to be defined in the beginning of the script
They may have or not arguments
Arguments and return values can be of any type
The variables defined inside the function are GLOBAL. They have to be defined LOCAL, if we want

#92

Backup: Functions: Example

Original slide 93

```
#!/bin/bash
outside="a global variable"
function mine() {
  local inside="this is local"
  echo $outside
  echo $inside
  outside="a global with new value"
}
echo $outside
mine
echo $outside
echo $inside
```

- ▶ a global variable
- ▶ a global variable
- ▶ this is local

